Homework 10

Yuchuan Xue

Due date: NOV 13, 2026 at 11:59 PM

Question 1.1 (5 points)

1.1.A

I trained the supplied randomly initialized 160M-parameter GPTNeoX model for one epoch on the pre-tokenized WikiText training split (115,503 sequences of length 512; 59,137,536 tokens), using eager attention and BF16 on one NVIDIA GeForce RTX 4080 SUPER. The per-device batch size was 16, with 2 gradient accumulation steps (effective batch size 32). I used AdamW with peak learning rate 6×10^{-4}, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, 1% warmup, cosine decay, gradient clipping at 1.0, and seed 42. Gradient checkpointing was disabled.

Metric	Measured value
Validation perplexity, length 512	39.93
Validation perplexity, length 2048	137.58
Training time	1594.79 s (26.58 min)
Peak training VRAM, PyTorch allocated	13124.14 MiB

Training time is the Hugging Face Trainer runtime, including epoch-end validation at length 512, and excluding model saving and the two final validation runs. Peak allocated memory was captured before the final validations; peak reserved CUDA memory over that interval was 14132.00 MiB. The maximum device VRAM sampled by `nvidia-smi` once per second over the full run, including final evaluation, was 14,456 MiB. All memory values are in MiB.

1.1.B

The perplexity on the 2048-token set is higher (137.58 versus 39.93). This is consistent with a mismatch between training and evaluation context lengths. During training, the model sees positions 0–511 and relative attention distances up to 511. Evaluation at length 2048 introduces larger positions and relative distances that were not represented in training. Although rotary position embeddings (RoPE) can be evaluated at these positions, the learned attention patterns need not extrapolate reliably. The additional context therefore does not guarantee lower perplexity for a model trained only at length 512. Continuing training at length 2048 would expose the model to the longer-range attention patterns used at evaluation.

Question 1.2 (5 points)

1.2.A

I completed `GPTNeoXFlashAttention2.forward()` in the supplied custom model. Q/K/V are transposed from [batch, heads, sequence, head dimension] to [batch, sequence, heads, head dimension]. The helper `_flash_attention_forward` receives the padding mask, causal-alignment flag and train/eval dropout. Its output heads are merged before the final projection.

I wrote 36 CUDA tests, all of which passed with no skips on an RTX 4080 SUPER using PyTorch 2.4.1 and FlashAttention 2.6.3. Small two-layer models with identical weights compare eager and flash logits, losses and every parameter gradient in FP16/BF16, with an additional FP32 eager reference. Coverage includes batch sizes 1/2, lengths 1/17/512/2048, left/right padding, strict causal non-leakage, and single/multiple-token cached decoding. Separate checks cover train/eval dropout and a real 160M-checkpoint forward/backward pass. Consistency comparisons disable dropout and use dtype-dependent tolerances; the causal-prefix check requires exact equality. Gradient comparisons use relative L2 and normalized maximum errors to handle cancellation near zero.

1.2.B

I trained from the same supplied initialization for one epoch, keeping all Q1.1 hyperparameters and the same GPU model. Only the attention backend was changed to FlashAttention 2; gradient checkpointing remained disabled.

Metric	Q1.1: eager	Q1.2: flash
Validation perplexity, length 512	39.93	39.97
Validation perplexity, length 2048	137.58	137.02
Training time (seconds)	1594.79	978.28
Peak training allocated VRAM (MiB)	13124.14	9682.39

The flash run took 16.30 minutes. Timing and memory definitions match Q1.1: Trainer runtime includes epoch-end 512-token validation and excludes model saving and the final validations; the allocated-memory peak is captured before those final validations. Flash peak reserved CUDA memory was 10614 MiB. The device-memory peak sampled once per second over the full flash run was 15,654 MiB.

1.2.C

The perplexities are similar across the two attention backends. FlashAttention computes the same mathematical attention operation, while floating-point reduction order and low-precision rounding can change individual updates and the resulting training trajectory. The much higher 2048-token perplexity persists because both models were trained only at length 512.

Flash training was 1.63 times as fast, reducing measured runtime by 38.7%. Peak allocated training memory fell by 26.2%.

The full-run device high-water mark was higher (15,654 versus 14,456 MiB), with the flash peak occurring at the end of the run. This measure includes unused allocator blocks and covers final evaluation, while the training tensor-allocation peak is captured beforehand.

FlashAttention tiles Q/K/V computation and performs online softmax using on-chip memory, avoiding the storage of a complete sequence-by-sequence attention matrix in GPU global memory. During backpropagation it recomputes attention intermediates from these tiles. This reduces attention-intermediate memory and global-memory traffic. Attention arithmetic remains quadratic in sequence length. FlashAttention-2 also improves GPU occupancy and work partitioning while reducing non-matmul work and inter-warp communication. These mechanisms explain potential speed and memory gains; other model operations and runtime overhead limit the end-to-end improvement. See Dao et al. (2022) and Dao (2023).

Question 1.3 (5 points)

1.3.A

On the RTX 4080 SUPER, I measured per-device batch limits using BF16, length 2048, the same Q1.2 checkpoint, accumulation 1, and a fresh process for every candidate. Each successful candidate completed three actual Trainer optimizer steps, including optimizer-state allocation; generation KV caching was disabled in both settings.

Setting	Largest passed batch	Next batch (OOM)
Eager, checkpointing off	2	3
FlashAttention 2, checkpointing on	9	10

These limits describe this fixed-shape three-step training probe. For the complete runs I used batch 4 with 2 accumulation steps to retain memory headroom.

1.3.B

Gradient checkpointing keeps selected activations from the forward pass and recomputes missing intermediates during backpropagation. It reduces activation memory at the cost of additional computation, which can make training slower (linked reference).

1.3.C

I continued from Q1.2 for one epoch on 28,531 length-2048 WikiText sequences (58,431,488 tokens), using FlashAttention 2, non-reentrant checkpointing and BF16. Batch 4 and accumulation 2 give 16,384 tokens per update; I used peak learning rate 10^{-4}, 1% warmup, cosine decay, AdamW, and seed 42. The matched scratch control uses the same settings.

Metric	Continued from Q1.2	From scratch
Validation perplexity, length 512	38.75	214.48
Validation perplexity, length 2048	43.84	230.34
Training time (seconds)	1162.22	1162.33
Peak allocated training VRAM (MiB)	6218.60	6218.60

Timing includes epoch-end validation and excludes saving/final validations, as in Q1.1/Q1.2. Allocated training peaks are captured before final validation. Reserved peaks were 7828/7828 MiB, and whole-run device sampling peaks were 8150/8150 MiB (continued/scratch).

1.3.D

The 2048-token perplexity decreased from 137.02 in Q1.2 (137.58 in Q1.1) to 43.84. The model now trains on the longer positions and relative attention distances used at evaluation, reducing the context-length mismatch; additional WikiText training also improves its predictions.

1.3.E

The scratch model starts from the supplied initialization, with the same 2048-token data, hyperparameters and one-epoch budget as the continuation run. It reached perplexities 214.48/230.34, compared with 38.75/43.84 for continuation (512/2048). The lower continuation perplexities are consistent with the benefit of the language-model knowledge already learned in Q1.2. The continued model has also received the earlier 512-token training in Q1.2, giving it more total training exposure and compute; this comparison does not isolate curriculum effects at equal total training budget.